

VOLUME 6 • B.TECH CSE STUDY SERIES

JAVASCRIPT (ES6+)

Complete Professional Handwritten Notes

V8 Engine, Execution Context, Closures, DOM Manipulation, Event Loop, Async/Await, Promises & Modern ES6+ Architecture

```
const fetchTech = async () => {  
  const res = await fetch('https://api.github.com/search/repositories?q=javascript');  
  return res.json();  
}; // Event Loop
```

✓ Execution Context & Call Stack

✓ Hoisting, Scope & Closures

✓ Event Loop, Microtasks & Macrotasks

✓ DOM Manipulation & Event Bubbling

✓ Promises & Async/Await Architecture

✓ 30+ Viva & SDE Interview Q&A

Designed for University Exams, Full-Stack Roles & Top Product Company Placements



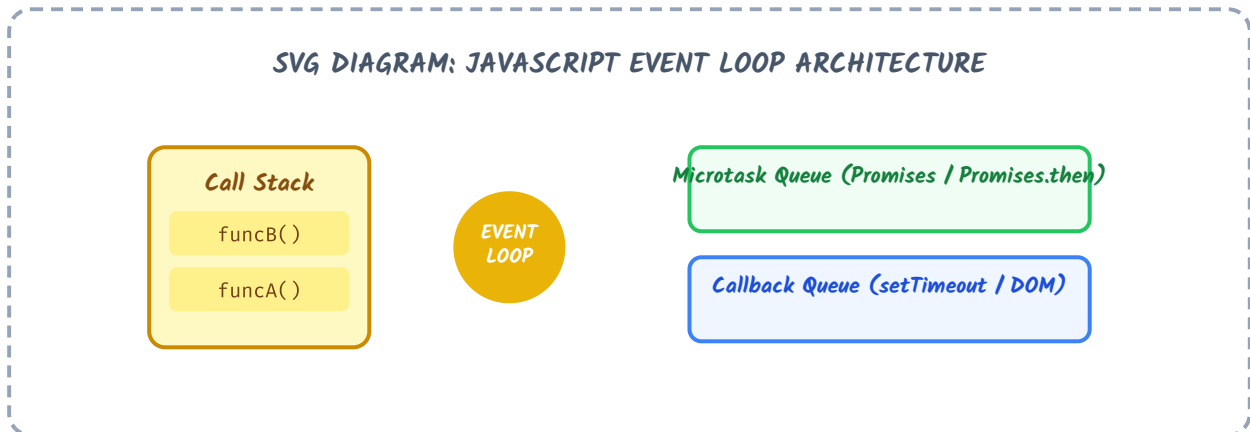
TABLE OF CONTENTS — JAVASCRIPT

Ch 1. JavaScript Engine (V8) & Execution Context	Page 02
Ch 2. Variables (var vs let vs const) & Data Types	Page 06
Ch 3. Functions, Arrow Functions & Higher-Order Functions	Page 11
Ch 4. Scope Chain, Hoisting & Temporal Dead Zone (TDZ)	Page 16
Ch 5. Closures & Practical Encapsulation	Page 21
Ch 6. Arrays & Array Methods (map, filter, reduce)	Page 26
Ch 7. Objects, Prototypes & Object-Oriented JS	Page 32
Ch 8. DOM Tree Manipulation, Events & Delegation	Page 38
Ch 9. Asynchronous JS: Callbacks, Promises & Async/Await	Page 45
Ch 10. The Event Loop, Call Stack & Task Queues	Page 52
Ch 11. Modern ES6+ Features (Destructuring, Modules)	Page 58
Ch 12. Comparison Tables & Visual Mind Maps	Page 63
Ch 13. Real-World Practical Projects (Weather API App, To-Do)	Page 68
Ch 14. Practice Problems (Basic → Advanced)	Page 74
Ch 15. 30+ University Viva Questions & Answers	Page 78
Ch 16. 30+ SDE Technical Interview Questions	Page 83
Ch 17. 1-Day Exam Cheat Sheet & JS Quick Reference	Page 88

CHAPTER 10: THE JAVASCRIPT EVENT LOOP & RUNTIME

💡 Call Stack, Web APIs, Microtasks & Macrotasks

SVG DIAGRAM: JAVASCRIPT EVENT LOOP ARCHITECTURE



⚡ **Priority Order:** Call Stack executes first → Then ALL Microtasks (Promise handlers) are drained → Then 1 Macrotask (setTimeout callback) is executed!

```
console.log("1. Start");

setTimeout(() => {
  console.log("4. Timeout (Macrotask)");
}, 0);

Promise.resolve().then(() => {
  console.log("3. Promise (Microtask)");
});

console.log("2. End");
```

Output: 1. Start 2. End 3. Promise (Microtask) 4. Timeout (Macrotask)

CHAPTER 12: COMPARISON TABLES

var vs let vs const

Feature	var	let	const
Scope	Function Scope	Block Scope	Block Scope
Hoisting	Hoisted with undefined	Hoisted in Temporal Dead Zone	Hoisted in Temporal Dead Zone
Re-declaration	Allowed	Not Allowed	Not Allowed
Re-assignment	Allowed	Allowed	Not Allowed

CHAPTER 13: REAL-WORLD PRACTICAL PROJECT

Project: Asynchronous Weather API Dashboard

```
async function fetchWeather(city) {
  const apiKey = "demo_key";
  const url = "https://api.openweathermap.org/data/2.5/weather?q=" +
city + "&units=metric&appid=" + apiKey;

  try {
    const response = await fetch(url);
    if (!response.ok) throw new Error("City not found");
    const data = await response.json();

    console.log("City: " + data.name);
    console.log("Temp: " + data.main.temp + "°C");
    console.log("Weather: " + data.weather[0].description);
  } catch (error) {
    console.error("Fetch Error:", error.message);
  }
}
```

CHAPTER 15: 30+ VIVA & INTERVIEW QUESTIONS

Q1. What is a Closure in JavaScript?

Ans: A closure is a function bundled together with references to its surrounding lexical environment. It gives an inner function access to an outer function's scope even after the outer function has returned.

Q2. What is Event Delegation?

Ans: Event delegation is a pattern where a single event listener is attached to a parent element instead of multiple child elements, taking advantage of event bubbling.